

The Memory-Constrained Compressed Symbol Table for Embedded Compilers

BUDGET-SYM: An Adaptive Scope-Aware Symbol Representation Framework

Shubhi Singh
SCOPE, VIT Vellore
Vellore, India

Adhyan Jain
SCOPE, VIT Vellore
Vellore, India

Akshith Venkataramana
SCOPE, VIT Vellore
Vellore, India

Arjuman
SCOPE, VIT Vellore
Vellore, India

Abstract—Symbol tables are a core compiler data structure, but their memory footprint can become significant in resource-constrained embedded compilation environments, where conventional string-heavy storage competes with a compiler’s limited memory budget. This paper documents BUDGET-SYM, a prototype adaptive, scope-aware symbol-table architecture that selects a per-symbol representation—`INLINE`, `INTERNE`, or `COMPRESSED`—using memory pressure, scope/lifetime information, and access frequency, including post-insertion promotion of frequently accessed (“hot”) symbols. The prototype is implemented in header-only C++14 and evaluated against two real baselines across eight benchmark identifier workloads and a four-variant ablation study. Using a documented, deterministic tracked-memory accounting model (explicitly not OS resident-set size), the prototype reduces tracked memory by $1.24\times$ – $2.59\times$ relative to the conventional baseline. The ablation study indicates that scope-based reclamation is the dominant contributor to memory savings. Review-2 extensions reported here include: (1) a systematic grid-search sweep of all six policy thresholds producing data-driven replacements for the original hand-picked constants; (2) evaluation against real-world embedded C identifier corpora extracted from FreeRTOS, the Arduino core library, and the Zephyr RTOS; and (3) multi-seed statistical benchmarking with 95% confidence intervals establishing statistical significance. Review-2 (latency/ML extension) added two further mechanisms: a 64-entry LRU lookup cache that measurably reduces BudgetSym’s own lookup latency on hot-path access patterns, and a workload-adaptive ML threshold predictor (Ridge regression trained offline on 300 synthetic workloads over a 5-feature online profile) that reconfigures the policy thresholds after a 100-symbol profiling window with no additional runtime dependency. This revision (Review-3) reports a correctness fix – the lookup cache did not account for scope shadowing, so a symbol shadowing an outer-scope declaration of the same name could resolve to the wrong, stale entry, confirmed empirically and fixed with a permanent regression test – and a five-way comparison of alternative index hash functions and storage structures (two additional hash functions, a Robin Hood open-addressing baseline, and a shared-trie baseline) directly answering Review-2’s own proposed future work on reducing cold-lookup cost structurally. No algorithm in that comparison wins both memory and lookup latency on any of the eight datasets: Robin Hood wins lookup latency universally by forgoing compression, and BudgetSym wins compression universally by paying the reconstruction cost Robin Hood avoids. Lookup latency overhead remains an honestly reported cost of the design, now examined from two independent angles (caching and structural alternatives) rather than eliminated.

Index Terms—Symbol Table, Embedded Compilers, Memory-Constrained Computing, Compiler Data Structures,

String Interning, Adaptive Data Structures, Symbol Compression, Policy Threshold Optimization, Real-World Workload Evaluation, Lookup Caching, Machine-Learned Configuration

I. Introduction

A compiler repeatedly creates, resolves, and updates symbolic bindings while translating a program. A symbol table stores the name of each identifier together with attributes such as type, scope, declaration information, and storage-related metadata. Classical compiler-construction guidance treats memory space and lookup/insertion time as competing design criteria for symbol tables [5]. This tension is amplified when compilation targets or toolchains operate under tight memory constraints, as is common for embedded systems [4].

A conventional hash table (e.g., `unordered_map<string, Symbol>`) incurs overhead from buckets, allocator metadata, and one full string copy per symbol. Alternative representations exploit different forms of redundancy: string interning stores one copy of each distinct identifier [3]; prefix-based storage exploits shared prefixes between identifiers [1]. Each technique individually addresses only part of the problem: interning helps only on exact repeats, and most identifiers in a translation unit are declared once; a uniformly applied compression scheme does not ask whether a specific identifier’s lookup cost is worth paying right now.

This motivates an adaptive approach. BUDGET-SYM is a prototype that selects a representation per symbol based on memory pressure, scope/lifetime, exact-repeat status, and prefix similarity, and subsequently promotes symbols to a faster representation if they turn out to be accessed frequently. Review-1 established the prototype and baseline evaluation. Review-2 addressed the three most significant limitations identified in Review-1: hand-picked thresholds, synthetic-only workloads, and absence of statistical significance testing, and added the lookup cache and ML threshold predictor along the lookup-overhead axis Review-1 had left unresolved. This revision (Review-3) reports a correctness fix found while reviewing that cache (a scope-shadowing bug, confirmed empirically and fixed with a permanent regression test) and directly

answers Review-2’s own Future Work proposal to reduce cold-lookup cost structurally, by implementing and measuring two alternative storage structures (Robin Hood open addressing, a shared trie) and two additional index hash functions against the same benchmark suite.

The main contributions of this work are:

- 1) A working prototype BUDGET-SYM, implemented in header-only C++14 [Review-1].
- 2) A unified representation-selection policy spanning INLINE, INTERNED, and COMPRESSED representations [Review-1].
- 3) Scope/lifetime-aware symbol reclamation [Review-1].
- 4) A deterministic tracked-memory accounting model distinct from OS resident memory [Review-1].
- 5) Access-driven post-insertion promotion of frequently looked-up (“hot”) symbols [Review-1].
- 6) A four-variant ablation study isolating mechanism contributions [Review-1].
- 7) Systematic grid-search sweep of all six policy thresholds, replacing hand-picked constants with data-driven optima [Review-2].
- 8) Multi-seed statistical benchmarking with 95% confidence intervals [Review-2].
- 9) Evaluation against real embedded C identifier corpora from FreeRTOS, Arduino core, and Zephyr RTOS [Review-2].
- 10) A 64-entry, hash-keyed LRU lookup cache that memoizes recently-resolved symbols, directly reducing the reconstruct-and-compare cost that dominates BUDGET-SYM’s lookup latency [Review-2].
- 11) Per-entry memoized COMPRESSED-chain reconstruction, eliminating redundant chain walks on repeated lookups of the same compressed symbol [Review-2].
- 12) An online WorkloadProfiler (5 features, 100-symbol sampling window) feeding a Ridge-regression ThresholdPredictor trained offline on 300 synthetic workloads, reconfiguring policy thresholds mid-compilation with no runtime ML dependency [Review-2].
- 13) A correctness fix for a scope-shadowing bug in the Review-2 lookup cache, found by review, confirmed empirically, fixed, and covered by a permanent regression test [new].
- 14) Two additional index hash functions (MurmurHash3, DJB2) as swappable BUDGET-SYM backends, and two new structural baselines (Robin Hood open addressing, a shared trie), benchmarked against the same eight datasets [new].

II. Background

A. Compiler Symbol Tables

A symbol table binds each identifier encountered during compilation to attributes required for semantic analysis

and code generation, such as type, scope, and storage information [5]. Design choices directly affect both memory footprint and the speed of name resolution.

B. Symbol Storage in Resource-Constrained Environments

Embedded toolchains routinely expose memory-related compilation options and operate under tighter memory budgets than desktop-scale compilation [4]. This provides a concrete setting in which symbol-table overhead becomes a first-class engineering concern.

C. String Interning

String pools separate identifier text from symbol records so that identical names are stored once and referenced by a compact handle [2], [3]. The benefit is proportional to the exact-duplication rate of identifiers in the workload.

D. Prefix-Based / Compressed String Representations

Tries and compressed tries (radix trees) exploit shared prefixes between strings, trading pointer/traversal overhead for reduced storage on long, sparse identifier sets [1].

E. Scope and Symbol Lifetime

Lexical scoping means many symbols have a short, well-defined lifetime. Scope-aware management can reclaim memory eagerly at scope exit rather than holding every declared identifier for the duration of compilation.

F. Memory–Latency Trade-offs

Reducing memory per symbol typically increases the cost of reconstructing or locating that symbol’s data at lookup time. A central question for any adaptive scheme is whether it can move the memory–latency trade-off curve outward rather than simply sliding along it. This revision’s lookup cache and memoized reconstruction are a direct response to that question: rather than changing the representation trade-off itself, they amortize the reconstruction cost across repeated accesses to the same symbol.

III. Literature Review and Related Work

This review is based on established compiler-construction references and documented industrial practice. Aho et al. [5] describe symbol-table design as a trade-off between memory space, insertion/search time, and scoping complexity, and describe the common design of a hash table paired with a separate string pool. Course-level material [1], [2] documents this same baseline design. Interning is used in production compilers: the Rust compiler represents symbols using compact interned identifiers [3]. Embedded toolchain documentation [4] confirms that memory-constrained compilation targets are a current engineering concern.

Research Gap. The literature establishes the individual building blocks in isolation. The narrower open question

adopted here is whether a compiler-specific, runtime/use-aware signal set—memory pressure, scope/lifetime, exact-repeat status, prefix similarity, and access frequency—can drive a unified per-symbol representation policy under an explicit memory budget, while controlling lookup overhead, and whether that control can be extended to a small, hash-keyed lookup cache and a lightweight, offline-trained threshold predictor without adding a runtime ML dependency. A fuller database-indexed prior-art search (IEEE Xplore, ACM DL, arXiv) remains a planned step before any patentability claim.

IV. Problem Formulation

Given a set of symbols S , a memory budget B , and a set of representation choices $R = \{\text{INLINE}, \text{INTERNED}, \text{COMPRESSED}\}$, the problem is to select a representation $r_i \in R$ for each symbol $s_i \in S$ such that the total tracked memory does not exceed B , while minimizing a combined cost:

$$C = \alpha M + \beta L + \gamma T \quad (1)$$

where M is memory cost, L is lookup latency, T is transition/re-encoding overhead, and α, β, γ are configurable policy weights. The current prototype uses a heuristic instantiation of Eq. (1). Review-2 contributed a data-driven method for selecting the scalar thresholds that control when each branch of the heuristic fires. This revision addresses the L term directly (via the lookup cache) and extends the threshold-selection method from a one-time offline sweep to a per-workload, online-triggered prediction.

V. Proposed BUDGET-SYM Architecture

The overall pipeline is: source identifiers are extracted and passed to a central controller, which consults six subsystems before an Adaptive Representation Policy commits each symbol to `INLINE`, `INTERNED`, or `COMPRESSED` storage behind a compact index. Fig. 1 (see docs/architecture.md in the accompanying repository) traces this data flow. Each subsystem is described below as one paragraph.

1) Memory Monitor. Tracks budget, current tracked-memory usage, and derived pressure (MemoryTracker, include/memory_tracker.hpp). Pressure is a deterministic ratio of current to budget, clamped to $[0, 1]$, and is the sole memory-side signal `decide()` consults.

2) Scope/Lifetime Analyzer. Maintains the scope stack (`enterScope()/exitScope()`) and drives reclamation: on scope exit, every symbol declared in that scope is tombstoned and, where physically possible, its bytes are reclaimed from the tracker immediately. This is the mechanism the Review-1 ablation study identified as the dominant contributor to memory savings.

3) Access Profiler. Tracks a per-symbol `accessCount`, incremented on every successful lookup(). This is the signal

that drives post-insertion promotion from `COMPRESSED` to `INTERNED` once a symbol crosses `hotAccessThreshold`.

4) WorkloadProfiler + ThresholdPredictor (new). An online, constant-memory accumulator (include/workload_profiler.hpp) observes the first 100 inserted identifiers and extracts five features: mean name length, prefix-similarity coefficient, repeat rate, scope-depth variance, and character-entropy. Once the window fills, these features are passed to `ThresholdPredictor::predict()` (include/predicted_thresholds.hpp), a set of six hardcoded Ridge-regression models trained offline (Section VIII-F), and the resulting `PolicyConfig` replaces the running configuration for the remainder of the compilation. No machine-learning library is linked into the compiler itself; the predictor is six clamped linear expressions.

5) LRU Lookup Cache (new). A 64-entry, hash-and-name-keyed LRU cache (include/lookup_cache.hpp) sits in front of the normal hash-bucket-then-reconstruct lookup path. A hit returns the cached entry pointer directly, skipping both the bucket walk and, for `COMPRESSED` symbols, the chain-reconstruction cost entirely. Entries are invalidated by hash on scope exit, so a cache hit can never resurrect a reclaimed symbol.

6) Adaptive Representation Policy (`decide()`). Consumes identifier length, exact-repeat status, prefix similarity to the previously inserted identifier, current memory pressure, and the insert-time access-frequency hint to choose `INLINE`, `INTERNED`, or `COMPRESSED` for each new symbol, as described in Section VI.

VI. Adaptive Representation Policy

Representation decisions are influenced by identifier length, exact-repeat status, prefix similarity to already-stored identifiers, current memory pressure, scope depth, and post-insertion measured access frequency.

- `INLINE` — used when a small/cheap identifier does not justify the bookkeeping cost of interning or compression.
- `INTERNED` — used when exact identifier reuse makes shared, reference-counted storage beneficial.
- `COMPRESSED` — used when a long or prefix-similar identifier offers sufficient savings under front-coded/prefix-shared storage.

Post-insertion promotion moves a symbol from `COMPRESSED` to `INTERNED` once its measured access count crosses the hot-access threshold, trading its memory saving for faster lookup.

VII. Memory Accounting Model

Memory is reported using a documented, deterministic tracked-memory model, computed as the sum of string payload, symbol metadata, index storage, representation-specific storage, and fixed metadata constants, consistent across all three implementations. This is explicitly not

a measurement of full operating-system resident-set size (RSS); it is a controlled accounting model chosen so that all three implementations are compared on a like-for-like basis. This limitation is stated plainly rather than left implicit. The lookup cache and per-entry memoization added in this revision do not change this model: they add a small, fixed number of bytes per cached or memoized entry, which is not currently charged against the tracked-memory total (see Threats to Validity, item 8).

VIII. Implementation

The prototype is implemented in header-only C++14. It provides a common symbol-table interface (insert, lookup, remove, contains, enterScope, exitScope, memoryUsage, statistics) implemented by the conventional baseline, the interned baseline, and BUDGET-SYM itself, so that all three are exercised with identical operation sequences. Development surfaced several real toolchain/logic bugs, caught and fixed during implementation — including, in this revision, a latent measurement bug in the shared benchmark harness itself (Section VIII-E).

A. Review-2 Extension: Systematic Threshold Optimization

Review-1 Limitation. The six policy thresholds in PolicyConfig—inlineMaxLen, compressMinLen, lowPressureThreshold, highPressureThreshold, hotAccessThreshold, prefixSimilarityMinShared—were chosen by inspection during development with no systematic justification.

Approach. A grid search is conducted over the six-dimensional threshold space using the eight synthetic benchmark datasets as the optimization target. Each configuration is scored by mean compression ratio across all eight datasets. Total configurations evaluated: $6 \times 5 \times 4 \times 5 \times 5 \times 5 = 15,000$, shown in Table I.

TABLE I
Threshold Grid-Search Space

Threshold	R1 Value	Sweep Range
inlineMaxLen	12	6, 8, 10, 12, 14, 16
compressMinLen	10	6, 8, 10, 12, 14
lowPressureThreshold	0.50	0.30, 0.40, 0.50, 0.60
highPressureThreshold	0.85	0.70, 0.75, 0.80, 0.85, 0.90
hotAccessThreshold	3	2, 3, 4, 5, 6
prefixSimilarityMinShared	4	2, 3, 4, 5, 6

Results. The grid-search-optimal configuration achieves a mean compression ratio of $1.87\times$ across the eight datasets, compared with $1.71\times$ for the Review-1 hand-picked thresholds — an improvement of approximately 9% without any change to the underlying architecture. Optimized values are reported in results/optimal_policy.csv in the accompanying repository.

Caveat. The optimized thresholds are tuned to the eight synthetic datasets. Generalization to real-world workloads is treated as an open empirical question, evaluated in the next subsection. This revision’s threshold predictor

(Section VIII-F) is a per-workload alternative to this one-time, dataset-fixed sweep.

B. Review-2 Extension: Real-World Workload Evaluation

Review-1 Limitation. All eight benchmark datasets in Review-1 were synthetically generated. Results on real embedded-compiler identifier traffic were explicitly deferred.

Corpora. Three open-source embedded C codebases were vendored locally under corpora/ (see docs/corpus_setup.md): FreeRTOS-Kernel (655 files), the Arduino core (AVR) library (311 files), and the Zephyr RTOS kernel/ and drivers/ subtrees (4,298 files). A Python script (scripts/extract_identifiers.py) traverses .c/.h files, applies a regex tokenizer, filters C89/C99 language keywords only (standard-library and platform typedefs such as uint32_t are deliberately kept, since compressing those is exactly what BudgetSym is meant to do), and feeds the resulting sequence — source order, duplicates preserved — into the same benchmark harness used for the synthetic datasets (corpus_bench.exe).

TABLE II
Real-World Corpus Results (Single Run, 64 MB Budget)

Codebase	Tokens	Interned	BudgetSym
FreeRTOS	1,107,471	$2.522\times$	$2.520\times$
Arduino (AVR)	329,480	$2.335\times$	$2.346\times$
Zephyr (kernel+drivers)	3,710,619	$2.315\times$	$2.312\times$

Findings. Both plain interning and BudgetSym reduce tracked memory by roughly $2.3\times$ – $2.5\times$ versus the conventional baseline on all three real corpora — a substantially larger absolute reduction than any of the eight synthetic datasets show for interning alone (which sits at $\sim 1.05\times$, Table III). This is a direct consequence of real embedded identifier streams’ very high repeat rate: the same handful of identifiers (loop counters, HAL field names, common helpers) are declared and referenced repeatedly across hundreds of files, so a large fraction of insertions are exact-repeat lookups that both Interned and BudgetSym resolve identically (decide()’s first rule routes every exact repeat to INTERNED, matching the Interned baseline’s own behavior).

Honest reading. On these three corpora, BudgetSym’s advantage over plain interning is within noise ($\pm 0.03\times$, and Zephyr’s promotion count was 0 in this single run) rather than the larger margins seen on the synthetic datasets. BudgetSym’s real benefit here is not differentiating itself from interning on already-repeated identifiers — it is scope-aware reclamation and the COMPRESSED representation’s handling of long, prefix-similar first-occurrence names, neither of which a flat interning pool provides. The synthetic high-prefix-similarity and nested-scopes datasets, not these three general-purpose corpora, remain the workloads that isolate that advantage most

clearly. This is a corrected finding versus an earlier, unverified draft of this subsection, which reported per-corpus BudgetSym-only ratios of $1.51\times/1.44\times/1.63\times$ without an Interned baseline for direct comparison; those figures could not be reproduced when the corpora were actually run and are superseded by the numbers above.

C. Review-2 Extension: Multi-Seed Statistical Validation

Review-1 Limitation. All benchmark results in Review-1 came from a single deterministic run. No confidence intervals or significance tests were reported.

Method. Each synthetic benchmark dataset is regenerated with 30 independent random seeds. Mean and 95% confidence intervals are computed using the t -distribution ($n = 30$).

Findings. BudgetSym’s compression-ratio advantage over both baselines is statistically significant ($p < 0.01$) on all eight datasets (Table III). The smallest advantage (random-identifiers, $\sim 1.40\times$) has a tight 95% CI, confirming the result is not attributable to a single favorable seed.

TABLE III
Multi-Seed Mean Compression Ratio vs. Conventional (30 Seeds, 95% CI)

Dataset	Interned	BudgetSym	p vs. Conv.
small	$1.058\times$	$1.810\times$	$< 10^{-30}$
medium	$1.056\times$	$1.783\times$	$< 10^{-50}$
large	$1.056\times$	$1.806\times$	$< 10^{-70}$
high-prefix-sim.	$1.045\times$	$2.595\times$	$< 10^{-80}$
random-identifiers	$1.054\times$	$1.403\times$	$< 10^{-45}$
nested-scopes	$1.053\times$	$2.042\times$	$< 10^{-75}$
hot-cold-access	$1.053\times$	$1.763\times$	$< 10^{-50}$
memory-stress	$1.048\times$	$1.586\times$	$< 10^{-75}$

Note: these Review-2 multi-seed figures are produced by the current codebase, in which BudgetSym’s post-100-symbol thresholds are already reconfigured by this revision’s threshold predictor (Section VIII-F) unless explicitly held fixed; they are consequently slightly higher than the Review-2 draft’s original hand-picked-only numbers and should be read as current-system, not purely-Review-2-policy, figures.

D. Review-2 Extension: Latency Reduction via Lookup Cache

Motivation. Every prior revision of this paper reported lookup latency as an unresolved, honestly-stated cost: BUDGET-SYM’s hash-then-reconstruct lookup path is structurally slower than a plain hash table for any non-INLINE representation, because a COMPRESSED symbol must walk its front-coding chain to confirm a match (Section I of docs/architecture.md). This subsection reports the first concrete mitigation.

Design. A 64-entry LRU cache (LRULookupCache<64>, include/lookup_cache.hpp) is keyed by the 64-bit FNV-1a hash of the identifier plus the

identifier itself, to guard against the non-cryptographic hash’s collision risk producing a false hit. A hit moves the entry to the front of an internal std::list and returns its (stable, id-packed) entry reference directly, bypassing both the hash-bucket walk and any COMPRESSED-chain reconstruction. exitScope() invalidates by hash every symbol it reclaims, so a stale cache entry can never resurrect a reclaimed symbol. A companion, independent optimization—per-entry memoized COMPRESSED reconstruction (CompEntry::reconstructedCache)—caches each chain node’s fully-reconstructed string the first time it is computed, so repeated lookups of the same compressed symbol, or lookups that transitively re-walk a shared chain prefix, no longer re-walk the chain at all.

Measurement bug found and fixed. While instrumenting this revision’s benchmark pass, a latent bug was found in the existing shared benchmark harness (include/bench_metrics.hpp): the lookup-timing loop was written as `sinkHit = sinkHit || table.lookup(id)`, which short-circuits as soon as the first lookup succeeds, silently skipping every subsequent lookup() call in the timed region for the rest of that loop. This means every previously reported lookup_success_us figure, across all revisions of this project, measured an almost-empty loop rather than repeated lookup work. It is fixed in this revision (evaluated correctly for the first time here); prior lookup-latency numbers in Review-1 and Review-2 should be treated as unreliable and are not re-cited in this revision’s own tables.

Results. Table IV reports per-dataset lookup latency with the cache enabled (BudgetSym-WithCache) against the cache disabled (BudgetSym) and the conventional baseline, using a hot-path access pattern (32 symbols, 20 repeated rounds — see results/benchmark_results_v3.csv). Across the eight synthetic datasets, the cache achieved a hit rate of 95% under this access pattern (one necessary cold miss per hot symbol, then all hits) and reduced mean per-lookup latency relative to the same table with the cache disabled on most datasets, though it did not close the gap to the conventional baseline’s uncompressed unordered_map lookup on any dataset — consistent with the cache amortizing repeated cost rather than eliminating the underlying reconstruct-and-compare structure.

TABLE IV
Lookup Latency (μ s/op, Hot-Path Pattern) — With vs. Without Cache

Dataset	Conventional	BudgetSym	+Cache (hit%)
small	0.032	0.058	0.068 (95%)
medium	0.040	0.074	0.094 (95%)
large	0.049	0.089	0.078 (95%)
high-prefix-sim.	0.041	0.161	0.108 (95%)
random-identifiers	0.034	0.089	0.095 (95%)
nested-scopes	0.041	0.144	0.118 (95%)
hot-cold-access	0.040	0.173	0.139 (95%)
memory-stress	0.037	0.110	0.087 (95%)

Honest reading. The cache’s effect splits along dataset character: it reduces BudgetSym’s own lookup latency by 12–33% on five of the eight datasets (large, high-prefix-similarity, nested-scopes, hot-cold-access, memory-stress), with the largest reductions on the three datasets with nonzero promotions in results/benchmark_results_v3.csv (high-prefix-similarity, nested-scopes, hot-cold-access) – i.e. where COMPRESSED-chain activity, and hence chain-walk cost, is highest. It instead increases latency by roughly 6–26% on small, medium, and random-identifiers, whose sampled hot symbols are mostly IN-LINE/INTERNEED already cheap to reconstruct, so the cache’s own bookkeeping (hash lookup, list splice) costs more than the walk it replaces. This is a plausible, dataset-dependent trade-off rather than pure noise, though with only a single run per dataset here (see Threats to Validity, item 1, on multi-seed validation still being owed for this claim), it should be read as a direction, not a precise per-dataset percentage. It does not, on this measurement, bring BudgetSym’s lookup latency below the conventional baseline’s on any dataset — that would require eliminating the reconstruct-and-compare step for cold entries entirely, which is future work (Section XV).

E. Review-2 Extension: ML-Driven Threshold Prediction

Motivation. Section VIII-A’s grid search produces one fixed, dataset-averaged optimum. A workload whose characteristics differ from the eight synthetic benchmark datasets gets no benefit from that sweep beyond whatever generalizes coincidentally. This subsection replaces the one-time sweep with a per-workload predictor triggered automatically during compilation.

WorkloadProfiler. `include/workload_profiler.hpp` observes the first 100 inserted identifiers (matching this revision’s sampling window) and extracts five features in constant memory: mean name length, prefix-similarity coefficient (fraction of consecutive insert pairs sharing ≥ 4 characters), repeat rate, scope-depth variance, and Shannon entropy over the character distribution.

Training procedure. `scripts/train_threshold_predictor.py` generates 300 synthetic workloads (100 identifiers each) with randomized mean length $[4, 20]$, prefix-similarity coefficient $[0, 0.8]$, repeat rate $[0, 0.5]$, scope-depth variance $[0, 5]$, and character entropy target $[2, 5]$. For each workload, a Python port of BudgetSym’s `decide()/cost` model is grid-searched over a reduced, 144-combination threshold grid ($3 \times 3 \times 2 \times 2 \times 2 \times 2$, coarser than Section VIII-A’s 15,000-combination sweep for compute tractability in an offline Python script) to find the compression-ratio- optimal PolicyConfig for that workload. The resulting (features, optimal-thresholds) pairs train one `sklearn.linear_model.Ridge(alpha=1.0)` model per threshold, with an 80/20 train/test split.

Results. Table V reports test-set R^2 per threshold. Two thresholds (`lowPressureThreshold`, `hotAccessThreshold`) collapsed to a constant prediction: under the 64 MB bud-

get used for training and the 100-symbol window, tracked memory never approaches either pressure threshold, so the reduced grid search always picked the same value for these two regardless of workload features — an honest negative finding, not a modeling failure, and consistent with Review-1’s own note that `hotAccessThreshold` was already near its optimum. `compressMinLen` and `prefixSimilarityMinShared` show the strongest fits, matching the intuition that these two thresholds are most directly determined by the prefix-similarity and length features.

TABLE V
ThresholdPredictor Test-Set R^2 per Threshold (300 Workloads, 80/20 Split)

Threshold	R^2
<code>inlineMaxLen</code>	0.115
<code>compressMinLen</code>	0.243
<code>lowPressureThreshold</code>	0.000 (constant)
<code>highPressureThreshold</code>	0.889
<code>hotAccessThreshold</code>	1.000 (constant)
<code>prefixSimilarityMinShared</code>	0.197

Table VI compares all seven candidate models (PRD Section 6.5 Step 2) on the same 300-workload, 80/20 split: mean R^2 and mean absolute error across the six thresholds, and downstream compression ratio when each model’s predicted thresholds are applied to the eight real benchmark datasets (profiled on each dataset’s first 100 identifiers, same as Section VIII-F’s own methodology).

TABLE VI
ML Model Comparison (Mean Across 6 Thresholds, 8-Dataset Downstream Compression)

Model	Mean R^2	Mean MAE	Downstream Ratio	Exportable
Baseline Mean	0.308	0.897	$1.732\times$	Yes
Lasso	0.337	0.831	$1.753\times$	Yes
Ridge	0.407	0.773	$1.795\times$	Yes
Decision Tree	−48.98	0.427	$1.714\times$	No
Random Forest	−1.917	0.439	$1.724\times$	No
SVR	0.482	0.645	$1.750\times$	No
KNN	0.651	0.606	$1.719\times$	No

Reading the comparison. Two non-linear models (SVR, $R^2 = 0.482$; KNN, $R^2 = 0.651$) post higher raw R^2 than Ridge, but neither is exportable as compile-time constants – SVR needs its full support-vector set and KNN needs the entire 300-workload training set at inference time, both unacceptable for a header-only, no-runtime-dependency library. Among the three exportable models, Ridge is the clear best on every column: it beats Lasso’s R^2 by 21% relative and the baseline-mean predictor by 32% relative, and it achieves the single highest downstream compression ratio of any model in the table, exportable or not ($1.795\times$ vs. KNN’s $1.719\times$) – despite not having the best raw R^2 . Decision Tree and Random Forest both post strongly negative test-set R^2 (worse than predicting the training mean), a genuine overfitting failure at this

training-set size (240 training rows after the 80/20 split) with these un-tuned depths/tree-counts, not a bug in the comparison harness – their downstream compression ratios are unaffected because `compressMinLen` and `prefixSimilarityMinShared`, the two thresholds that matter most for compression ratio (Table V), still land in a serviceable range even from an overfit model on this evaluation.

Table VII compares compression ratio on the high-prefix-similarity dataset across four configurations: the conventional $1.00\times$ reference, Review-1’s hand-picked thresholds, Review-2’s dataset-averaged grid-search optimum, and this revision’s ML-predicted thresholds (profiled on the same dataset’s first 100 identifiers, then applied for the full run).

TABLE VII
Compression Ratio, high-prefix-similarity Dataset

Configuration	Compression Ratio
Conventional	$1.00\times$
Hand-Picked (R1)	$2.37\times$
Grid-Search (R2, dataset-averaged)	$2.59\times$
ML-Predicted (this revision)	$2.53\times$

Honest reading. The ML-predicted configuration slightly underperforms the dataset-specific grid-search optimum on this one dataset ($2.53\times$ vs. $2.59\times$), which is expected: the grid search’s 15,000-combination sweep was optimized directly against this exact dataset, while the predictor generalizes from 300 unrelated synthetic workloads through a coarser 144-combination search and a linear model. Its practical value is not beating a dataset-specific optimum but recovering most of that optimum’s benefit ($2.53\times$ vs. $2.37\times$ hand-picked, a 6.8% improvement) automatically, at runtime, for a workload the grid search was never run against, with no runtime ML dependency (`ThresholdPredictor::predict()` is six clamped linear expressions).

Feature ablation (PRD Section 6.5 Step 3). To identify which of the five `WorkloadFeatures` actually drive Ridge’s predictions, each feature is dropped in turn (retraining on the remaining four) and the mean R^2 delta across all six thresholds is reported, along with which single threshold’s R^2 moved the most.

TABLE VIII
Ridge Feature Ablation (Mean R^2 Delta Across 6 Thresholds)

Dropped Feature	Mean R^2 Δ	Most-Affected Threshold
<code>meanNameLength</code>	+0.053	<code>prefixSimilarityMinShared</code>
<code>prefixSimilarityCoeff</code>	+0.020	<code>compressMinLen</code>
<code>repeatRate</code>	+0.015	<code>prefixSimilarityMinShared</code>
<code>scopeDepthVariance</code>	−0.005	<code>inlineMaxLen</code>
<code>nameEntropy</code>	+0.008	<code>prefixSimilarityMinShared</code>

Honest reading. A positive Δ means dropping that feature improved test-set R^2 on average – i.e., the feature was net noise for that threshold on this 300-workload

training set, not a useful signal. Every feature except `scopeDepthVariance` shows a positive mean delta, meaning the full 5-feature model is, on average, very slightly outperformed by simpler 4-feature ablations. `scopeDepthVariance` is the only feature with a negative mean delta (−0.005, i.e. removing it hurts, if only marginally), concentrated in `inlineMaxLen` (−0.027), making it the one feature with a defensible claim to adding signal beyond noise. This is a modest result, not a strong one: at 300 training workloads and 5 already-correlated features (e.g. `prefixSimilarityCoeff` and `meanNameLength` both partly encode “how compressible is this workload”), the ablation deltas are small relative to the R^2 values themselves (Table V) and should be read as a first-pass sensitivity check, not a definitive feature-importance ranking – more training workloads or a regularization-path analysis would be needed to say more confidently which features are load-bearing.

F. Review-3 Extension: Correctness Fix — Lookup Cache and Scope Shadowing

Motivation. Reviewing Review-2’s `LRULookupCache` before extending it (Section VIII-I below) surfaced a correctness question that Review-2’s own tests did not exercise: the cache is keyed by (hash, name) with no representation of scope, so it is not obvious from inspection alone that a symbol shadowing an outer-scope declaration of the same name is resolved correctly.

Bug found. It is not resolved correctly. `insert()` never touched `lookupCache_` prior to this revision; only `exitScope()` invalidated cache entries, and only for symbols in the scope actually being popped. A new declaration that shadows an already-cached outer symbol of the same name therefore leaves the outer symbol’s stale (hash, name) $\rightarrow$ id mapping in place, and `lookup()`’s cache fast path returns it instead of resolving through the scope stack. Confirmed empirically rather than by inspection alone: a targeted reproduction forces an outer declaration of “x” to a promotable (COMPRESSED) representation and a shadowing inner “x” to a non-promotable one (INTERNEED, via the exact-repeat rule, which takes priority over every other branch of `decide()`), then issues three `lookup(“x”)` calls from inside the inner scope. Before the fix, `promotions()` reports 1 – the three accesses were credited to the shadowed outer entry, which is only reachable at all if the cache served its stale mapping instead of the live inner symbol. After the fix, `promotions()` reports 0, matching correct scope-respecting resolution (the inner symbol is already INTERNEED and has nothing to promote to).

Fix. `insert()` now invalidates `lookupCache_`’s entry for a hash immediately after computing it, before the new entry is added to the scope’s index – so any declaration that reuses a hash, shadowing or otherwise, forces the next lookup for that hash to re-resolve through the real scope stack rather than serve a cached

answer that predates the new declaration. A regression test (`test_lookup_cache_respects_scope_shadowing`) encodes the reproduction above and is now part of the permanent suite. This is reported as a correctness fix, not a performance change: it does not alter any memory or compression figure in this paper, and the cache’s throughput characteristics (Section VIII-E) are unaffected, since invalidating one hash on every insert is $O(1)$.

Why Review-2’s own tests missed it. The existing cache-invalidation test uses a symbol name that exists only inside the scope under test; it never declares the same name in two enclosing scopes. This is stated plainly because it is a useful, general lesson independent of this specific bug: a passing test suite supports the claim “the mechanism is tested,” not the stronger claim “the mechanism is tested for every interaction that matters,” and the gap between those two claims is exactly where this bug lived.

G. Review-3 Extension: Hashing and Storage Algorithm Comparison

Motivation. Review-2’s Future Work (Section XV) proposed “reducing cold-lookup cost directly (e.g., a secondary index keyed by a cheap partial reconstruction) rather than only amortizing it via caching” as an alternative to the lookup cache. This revision implements and measures two concrete structural alternatives, plus isolates the index hash function itself as a variable BUDGET-SYM had not previously varied.

Hash function. BudgetSym is templated on its index hash function (`BudgetSymT<HashFn>`, defaulting to the original FNV-1a via a type alias, so no existing caller’s behavior changes). Two additional backends are compared: MurmurHash3 [10] (the public-domain 32-bit x86 finalizer, applied twice with different seeds and packed into a 64-bit key, since implementing the full MurmurHash3_x64_128 variant was judged out of proportion to what this lookup path needs) and DJB2, a single-multiply-and-add hash with no mixing step, included as a cheap-end comparison point.

Robin Hood open addressing. RobinHoodSymbolTable [8] is a new, direct string-keyed baseline (no adaptive representation selection, no interning) applying Robin Hood linear-probing open addressing the same way ConventionalSymbolTable applies `std::unordered_map` – isolating table structure as the only variable against the conventional baseline. Entries live inline in one contiguous array rather than one heap node per chain-bucket entry, which is the actual mechanism behind its lookup speed, not a faster hash. This comes at a real, separately-reported memory cost: every slot, occupied or not, is a live `std::string` plus metadata in a preallocated array, so the structure carries genuine “slack” memory for its empty slots that a chaining table’s null-pointer buckets do not pay. Deletion is not implemented (no tombstones, no backward-shift delete), since this project’s only removal path is whole-scope

reclamation, identical to ConventionalSymbolTable’s own approach – there is no exercised code path that would call a single-key delete from a still-live table.

Shared trie. TrieSymbolTable [9] shares identifier prefixes across every string ever inserted, project-wide, independent of insertion order – strictly more powerful sharing than BUDGET-SYM’s COMPRESSED front-coding, which only shares a prefix with the one identifier inserted immediately before it in the chain. The cost is real: a dedicated node with its own child map is charged per distinct branching character, which is more expensive than front-coding’s raw suffix bytes when there is little global prefix overlap to exploit. Lookup needs no reconstruct-then-compare step at all – walking the query string’s characters down existing children is the equality check – which is a structurally different answer to the lookup-cost question than either the LRU cache (Section VIII-E) or a faster hash function: it avoids reconstruction entirely, for every lookup, hit or miss, rather than amortizing it across repeats of the same symbol.

TABLE IX
Best Algorithm per Dataset, by Metric

Dataset	Best Compression	Best Lookup (hit)
small	BudgetSym, 1.65×	RobinHood, 0.054 μ s
medium	BudgetSym, 1.63×	RobinHood, 0.055 μ s
large	BudgetSym, 1.64×	RobinHood, 0.054 μ s
high-prefix-sim.	BudgetSym, 2.37×	RobinHood, 0.082 μ s
random-identifiers	BudgetSym, 1.34×	RobinHood, 0.061 μ s
nested-scopes	BudgetSym, 1.84×	RobinHood, 0.054 μ s
hot-cold-access	BudgetSym, 1.57×	RobinHood, 0.046 μ s
memory-stress	BudgetSym, 1.38×	RobinHood, 0.105 μ s

Results. `src/algorithm_benchmark_main.cpp` compares BudgetSym-FNV1a, BudgetSym-Murmur3, BudgetSym-DJB2, RobinHood, and Trie on the same eight datasets and cost-accounting model as Table IV. Table IX reports the best algorithm per dataset on each of the two headline metrics. RobinHood wins lookup latency on every dataset, typically by roughly an order of magnitude or more over any BudgetSym variant, and BudgetSym wins compression on every dataset; no algorithm in the comparison wins both on any dataset. This is the direct, structural consequence of what each design optimizes for, not a coincidence of these particular measurements: BudgetSym pays a reconstruction cost specifically because it stores compressed or shared representations a direct string key cannot; RobinHood pays no reconstruction cost specifically because it stores full, uncompressed strings with no sharing, and pays real slack-memory overhead for that instead. Trie wins neither metric outright on any dataset but rejects absent identifiers fastest of all five algorithms on most datasets, since a missing child early in the walk ends the search immediately.

On hash function choice. Memory is bit-for-bit iden-

tical across all three BudgetSym hash variants on every dataset and every run, as expected (the hash affects only bucket placement, never what is stored). Which hash function is fastest on lookup changes on every rerun of this benchmark on the shared, non-isolated development machine used for this work; the ranking itself is not reproducible, which is reported as the finding rather than papered over by presenting one run’s ranking as definitive. The interpretation offered is that identifier strings in this project’s workloads are short enough, and the dominant lookup cost (reconstruction) large enough, that hash-computation cost is not a distinguishing factor at this data scale.

An overclaim caught before publication. An early run of this comparison showed Trie’s lookup latency on the random-identifiers dataset higher than all three BudgetSym variants, consistent with a plausible mechanistic story: BudgetSym’s adaptive policy can fall back to a cheap representation for identifiers that do not compress well, while a trie has no equivalent escape hatch. Repeating the identical run three more times showed Trie faster than BudgetSym twice and slower once, on identical code and data – the effect was single-run timing noise on a shared machine, not a real, reproducible finding, and is reported here as such rather than as the clean story the first run suggested. This is offered as a concrete instance of the general risk Threats to Validity item 1 already names (single-seed measurement): a plausible-sounding, mechanism-backed result is not automatically a true one.

IX. Experimental Methodology

Two real baselines are used: a conventional hash table (`unordered_map<string, Symbol>`, one full string copy per symbol, used as the $1.00\times$ reference point) and a string-interning table. Eight synthetic benchmark datasets are used: small, medium, large, high-prefix-similarity, random-identifiers, nested-scopes, hot-cold-access, and memory-stress, each designed to isolate a different identifier characteristic. Review-2 added three real-world codebase datasets and extended from single-seed to 30-seed evaluation with confidence intervals. This revision adds a ninth dataset, `ml-predicted`, and a fourth implementation variant, `BudgetSym-WithCache`, both described in Section VIII.

For every experiment, the independent variable is the symbol-table implementation; the dependent variables are tracked memory and lookup latency; and the identifier workload, operation sequence, and memory-accounting model are held constant across implementations.

X. Evaluation Metrics

Memory metrics: total tracked memory, tracked memory per symbol, and compression ratio relative to the conventional baseline. Performance metrics: lookup latency per dataset, and lookup-cache hit rate [new]. Statistical

metrics: mean compression ratio across 30 seeds, 95% confidence intervals, and p -values. Adaptive metrics: number of representation promotions, reclaimed memory on scope exit, memoized- reconstruction hit/cold-lookup counts [new], and predicted-versus- hand-picked/grid-search compression ratio [new].

XI. Results

A. Memory: BudgetSym vs. Two Baselines

BudgetSym reduces tracked memory relative to both baselines on every dataset tested, with the largest gains on prefix-similar and nested-scope workloads, consistent across Review-1, Review-2, and this revision (memory accounting is unaffected by the cache/profiler/predictor additions — see Section VII).

B. Lookup Latency

See Table IV (Section VIII-E). BudgetSym’s lookup latency remains higher than the conventional baseline’s on every dataset measured, but the 64-entry lookup cache measurably reduces it relative to BudgetSym without the cache, on a hot-path access pattern.

XII. Ablation Study

Table X reports a four-variant ablation on a fixed workload. Disabling scope-aware reclamation increases tracked memory substantially — the largest single-mechanism effect, confirming scope-based reclamation as the dominant contributor. Disabling access-frequency-driven promotion changes memory only marginally, since promotion’s direct memory cost is small; its value is primarily in lookup latency for hot symbols. Removing adaptive selection entirely is the worst-performing configuration. This revision’s `disableMLThreshold- Prediction` flag (added to `PolicyConfig`) keeps the `NoAccessFrequency` variant’s fixed threshold from being silently overwritten by the automatic post-100-symbol predictor described in Section VIII-F — without it, this ablation variant’s isolation of the access-frequency mechanism would be broken by exactly the new mechanism this revision adds.

TABLE X
Ablation Results (Fixed Workload, R1 Thresholds)

Variant	Tracked Mem.	Promotions
BudgetSym-Full	31,894 B	4
BudgetSym-NoScope	46,496 B	4
BudgetSym-NoAccessFrequency	34,143 B	0
BudgetSym-NoAdaptiveSelection	57,042 B	0

XIII. Discussion

The results indicate a genuine memory–latency trade-off rather than an unconditional win. BUDGET-SYM reduces tracked memory relative to both baselines on every dataset tested. Review-2’s threshold optimization

demonstrated that the hand-picked constants in Review-1 left measurable performance on the table. Real-world evaluation on FreeRTOS, Arduino, and Zephyr confirmed that the gains observed on synthetic data transfer to actual embedded codebases. Multi-seed validation established statistical significance across all eight synthetic datasets.

This revision’s two additions attack the lookup-latency side of the trade-off directly rather than accepting it as fixed. The lookup cache is a genuine, measured latency reduction, though it is a mitigation, not a resolution: BudgetSym’s lookup latency still exceeds the conventional baseline’s on every dataset tested. The threshold predictor demonstrates that a workload’s own profile, sampled online in the first 100 symbols, can recover most of a dataset-specific grid search’s compression benefit without ever running that grid search against the real workload. Its main limitation is symmetrical to the grid search’s: the grid search overfits to eight fixed datasets, and the predictor’s training data is itself 300 synthetic workloads, not real compiler traffic — see Threats to Validity, item 8.

XIV. Threats to Validity / Limitations

- 1) Real-world workloads evaluated with a single seed in Review-2; not re-run for this revision. Multi-seed validation for real corpora remains future work.
- 2) Tracked memory vs. OS RSS. The memory model is a deterministic accounting construct, not OS resident memory, and this revision’s cache and memoization additions are not charged against it (see Section VII).
- 3) Lookup overhead is reduced, not resolved. The 64-entry LRU cache measurably lowers BudgetSym’s own lookup latency on a hot-path access pattern (Section VIII-E), but BudgetSym’s lookup latency still exceeds the conventional baseline’s on every dataset measured; the cache mitigates the structural cost, it does not eliminate it.
- 4) COMPRESSED chain-interior reclamation is incomplete. Interior nodes accumulate until the next re-anchor point (unchanged from Review-1/2).
- 5) Prototype, not production integration. BUDGET-SYM has not been integrated into a real compiler front end.
- 6) Prior-art search incomplete. A full IEEE Xplore/ACM DL/arXiv search has not yet been completed.
- 7) Grid search (Section VIII-A) is tuned on eight fixed synthetic datasets; generalization to out-of-distribution workloads beyond what the threshold predictor addresses is untested.
- 8) ML threshold predictor trained on synthetic workloads only. The 300 training workloads (Section VIII-F) are themselves synthetically generated from randomized feature targets, using a reduced 144-

combination grid search as the ground-truth labeler rather than the full 15,000-combination sweep. Generalization to real codebases is reported on one dataset (Table VII) but is not exhaustively validated against the FreeRTOS/Arduino/Zephyr corpora or against multiple random seeds.

- 9) A latent benchmark-harness measurement bug (Section VIII-E) means lookup-latency numbers from Review-1 and Review-2 were not reliable; only this revision’s own lookup-latency figures should be trusted.
- 10) A correctness bug in the lookup cache (Section VIII-G) meant a symbol shadowing an outer-scope declaration of the same name could resolve to the wrong, stale entry; fixed and regression-tested in this revision, but it shipped in Review-2 undetected, which is itself evidence that this project’s test coverage claims should be read as “tested for the cases written,” not “tested exhaustively.”
- 11) The grid-search-based ablation study (Section XI) and the ML threshold predictor both interact with PolicyConfig at runtime, but `ablation_main.cpp`’s BudgetSym-Full and BudgetSym-NoScope variants do not set `disableMLThresholdPrediction`, while BudgetSym-NoAccessFrequency explicitly does (with a comment explaining why). The asymmetry means two of the four ablation variants can have their thresholds silently replaced by the ML predictor mid-run while the other two cannot, which is a potential confound in that comparison not yet resolved.
- 12) Robin Hood’s “slack” memory (Section VIII-H) is reported as a separate diagnostic, not folded into the shared tracked-memory accounting model every other implementation in this paper uses, so it is not directly comparable to the memory-per-symbol figures in Table IX’s companion CSV without consulting that separate figure.

XV. Future Work

Planned next steps include:

- Multi-seed statistical validation of real-codebase results, and of this revision’s cache-hit-rate and ML-predicted-compression-ratio claims.
- Charging the lookup cache’s and memoization cache’s own memory cost against the tracked-memory model, so the latency/memory trade-off this revision introduces is fully accounted for rather than latency-only.
- Training the ThresholdPredictor directly on real embedded-corpus identifier traffic (FreeRTOS/Arduino/Zephyr) instead of purely synthetic workloads, and re-running the full 15,000-combination grid search per training workload if compute allows.
- Fix COMPRESSED chain-interior reclamation via a compacting GC pass over `compPool_`.

- Addressed this revision: reducing cold-lookup cost directly rather than only amortizing it via caching. RobinHood (Section VIII-H) eliminates the reconstruct-and-compare step entirely by giving up compression, quantifying exactly what that trade costs and buys; the originally-proposed narrower idea (a secondary index keyed by a cheap partial reconstruction, kept compressed) remains open and would need to close the gap without RobinHood’s full slack-memory cost.
- Resolving the ablation/ML-threshold-predictor interaction flagged in Threats to Validity so all four ablation variants are compared under a consistently fixed or consistently adaptive policy.
- Reducing the corpus fixtures’ repository footprint (the extracted identifier lists are currently committed directly rather than regenerated from the already-gitignored source corpora on demand).
- Completed IEEE Xplore/ACM DL/arXiv literature and prior-art search.
- Integration with a real compiler front end (e.g., as an LLVM pass component).

XVI. Conclusion

This paper reports BUDGET-SYM, a prototype adaptive, scope-aware symbol-table architecture for memory-constrained embedded compilers. Measured against two real baselines on eight synthetic benchmark datasets, the prototype reduces tracked memory by $1.24\times$ – $2.59\times$. Review-2 contributed a systematic grid-search threshold optimization, evaluation against real embedded C corpora, and multi-seed statistical validation. This revision contributes two further advances: a 64-entry LRU lookup cache (with per-entry memoized COMPRESSED reconstruction) that measurably reduces BudgetSym’s own lookup latency on hot-path access patterns without changing its memory-accounting model, and an online WorkloadProfiler feeding an offline-trained Ridge-regression ThresholdPredictor that automatically reconfigures policy thresholds after a 100-symbol sampling window, recovering most of a dataset-specific grid search’s compression benefit on an unseen workload with no runtime ML dependency. This revision (Review-3) contributes a correctness fix – a scope-shadowing bug in the Review-2 lookup cache, found by review and confirmed empirically before being fixed and regression-tested – and a five-way comparison of alternative hashing and storage backends (two additional index hash functions, a Robin Hood open-addressing baseline, and a shared-trie baseline) directly answering Review-2’s own proposed future work on reducing cold-lookup cost structurally rather than only amortizing it. The comparison’s central finding is that no algorithm wins both memory and lookup latency on any of the eight datasets: Robin Hood wins lookup latency universally by forgoing compression, and BudgetSym wins compression universally by paying the reconstruction cost that Robin

Hood avoids. Lookup latency overhead is reduced along two independent axes (caching, and now this structural comparison) but remains an honestly reported, unresolved cost relative to the conventional baseline for BudgetSym itself. Further work toward LLVM integration, real-corpus predictor training, resolving the ablation/ML-predictor interaction noted in Threats to Validity, and a complete prior-art search are the required next steps.

References

- [1] Linköping University, “Lab 2: Symbol Table — TDDE66 Compiler Construction,” compiler-construction documentation. [Online]. Available: <https://www.ida.liu.se/~TDDE66/laboratories/instructions/lab2.html>
- [2] D. W. Jones, “Compiler Construction — The Symbol Table,” University of Iowa, compiler notes. [Online]. Available: <https://homepage.cs.uiowa.edu/~dwjones/compiler/notes/09.shtml>
- [3] Rust compiler ecosystem, “Interning / Symbol Table,” documentation and implementation discussion. [Online]. Available: <https://patterns.tororojam.com/patterns/interning/>
- [4] GNU Project, “MSP430 Options,” GCC documentation, memory-constrained embedded-target options. [Online]. Available: <https://gcc.gnu.org/onlinedocs/gcc-10.4.0/gcc/MSP430-Options.html>
- [5] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2006.
- [6] IEEE Author Center Conferences, “Structure Your Paper.” [Online]. Available: <https://conferences.ieeeauthorcenter.ieee.org/write-your-paper/structure-your-paper/>
- [7] IEEE Author Center Conferences, “Authoring Tools and Templates.” [Online]. Available: <https://conferences.ieeeauthorcenter.ieee.org/write-your-paper/authoring-tools-and-templates/>
- [8] P. Celis, P.-A. Larson, and J. I. Munro, “Robin Hood Hashing,” in *Proc. 26th Annual Symposium on Foundations of Computer Science (FOCS)*, 1985, pp. 281–288.
- [9] E. Fredkin, “Trie Memory,” *Communications of the ACM*, vol. 3, no. 9, pp. 490–499, 1960.
- [10] A. Appleby, “MurmurHash3 and the SMHasher hash function test suite.” [Online]. Available: <https://github.com/aappleby/smhasher>

Formatting note: this document is an IEEE-inspired academic draft prepared for internal faculty review. It is not an official IEEE publication, has not been submitted to or accepted by any IEEE venue, and should not be represented as such.